

■ ENVIRONNEMENT LOCAL COMPLET

ShipClass sur votre machine

Faire tourner Kubernetes, ArgoCD, JFrog, cert-manager,
Ingress NGINX et PostgreSQL entièrement en local

Windows · macOS · Linux — Zéro cloud, zéro coût, zéro Azure requis



Docker Desktop



kind ou minikube



ArgoCD



JFrog Artifactory



cert-manager



Ingress NGINX



PostgreSQL (Docker)



Grafana + Prometheus

Ce tutoriel vous guide pas-à-pas pour faire tourner l'intégralité du projet ShipClass en local sur votre machine : cluster Kubernetes, registry d'images, GitOps, certificats TLS auto-signés, base de données et monitoring. Aucun compte Azure ni carte bancaire requis.

PARTIE 1 — Prérequis : Ce qu'il faut installer

Liste de tous les outils à installer sur votre machine avant de commencer, avec les liens officiels et les commandes de vérification.
Compatibles Windows, macOS et Linux.

1.1 — Configuration Matérielle Recommandée

Ressource	Minimum	Recommandé	Pourquoi
RAM	8 GB	16 GB	kind + tous les pods (ArgoCD, JFrog, PostgreSQL, app) consomment ~6-8 GB
CPU	4 cœurs	8 cœurs	Les conteneurs en parallèle saturent rapidement 4 cœurs
Disque	30 GB	60 GB	Images Docker (JFrog ~2 GB, PostgreSQL ~400 MB, services ~200 MB chacun)
OS	Toute version récente	Windows 11, macOS 13+, Ubuntu 22.04+	Docker Desktop nécessite WSL2 sur Windows

1.2 — Outils à Installer

Docker Desktop	4.x+	docker.com/products/docker-desktop	Moteur de conteneurs + Kubernetes local (optionnel). Requis sur Windows/macOS. Sur Linux : Docker Engine suffit.
kubectkl	1.29+	kubernetes.io/docs/tasks/tools	CLI Kubernetes. Inclus avec Docker Desktop. Sinon : winget install Kubernetes.kubectkl
kind	0.23+	kind.sigs.k8s.io	Kubernetes IN Docker. Crée un vrai cluster K8s dans des conteneurs Docker. Léger et rapide.
Helm	3.14+	helm.sh/docs/intro/install	Gestionnaire de packages Kubernetes. Requis pour installer ArgoCD, JFrog, cert-manager, NGINX.
ArgoCD CLI	2.10+	argo-cd.readthedocs.io/en/stable/cli_installation	CLI pour interagir avec ArgoCD depuis le terminal.
Git	2.x+	git-scm.com	Gestion du code source. Requis pour ArgoCD (repo local ou GitHub).
mkcert	1.4+	github.com/FiloSottile/mkcert	Génère des certificats TLS locaux de confiance. Remplace Let's Encrypt en local.
PowerShell 7	7.4+	github.com/PowerShell/PowerShell	Sur Windows : déjà disponible. Sur macOS/Linux : brew install powershell / apt install powershell.
jq	1.7+	jqlang.github.io/jq	Outil CLI pour parser le JSON (utile pour les sorties kubectkl).
k9s (optionnel)	0.32+	k9scli.io	Interface TUI pour Kubernetes. Visualiser les pods, logs, events en temps réel dans le terminal.

1.3 — Installation rapide (une seule commande par OS)

■ Windows

Windows — PowerShell en tant qu'Administrateur (winget)

```
# Installer tous les outils d'un coup avec winget
winget install Docker.DockerDesktop
winget install Kubernetes.kubectl
winget install Helm.Helm
winget install Git.Git
winget install jqlang.jq # kind et mkcert via Scoop (gestionnaire de paquets recommandé)
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
Invoke-RestMethod https://get.scoop.sh | Invoke-Expression
scoop install kind mkcert # ArgoCD CLI
$version = (Invoke-RestMethod https://api.github.com/repos/argoproj/argo-cd/releases/latest).tag_name
Invoke-WebRequest "https://github.com/argoproj/argo-cd/releases/download/$version/argocd-windows-amd64.exe" -OutFile "$env:LOCALAPPDATA\Microsoft\WindowsApps\argocd.exe" # PowerShell 7 (si pas déjà installé)
winget install Microsoft.PowerShell # Activer WSL2 (requis pour Docker Desktop sur Windows)
wsl --install # Redémarrer la machine, puis ouvrir Docker Desktop
```

■ macOS

macOS — Homebrew

```
# Installer Homebrew si pas encore fait
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)" # Installer tous les outils
brew install kubectl helm kind git jq mkcert powershell/tap/powershell # Docker Desktop pour macOS
brew install --cask docker # Ou télécharger depuis docker.com (Apple Silicon M1/M2/M3 : choisir la version ARM64)
# ArgoCD CLI
brew install argocd # k9s (optionnel, très utile)
brew install derailed/k9s/k9s # Vérifier les installations
docker --version && kubectl version --client && helm version && kind version && argocd version --client
```

■ Linux

Linux — Ubuntu/Debian (bash)

```
# Docker Engine (sans Docker Desktop)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER
newgrp docker # kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
chmod +x kubectl && sudo mv kubectl /usr/local/bin/ # Helm
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash # kind
curl -LO kind https://kind.sigs.k8s.io/dl/v0.23.0/kind-linux-amd64
chmod +x kind && sudo mv kind /usr/local/bin/ # mkcert
sudo apt install libnss3-tools curl -Lo mkcert https://github.com/FiloSottile/mkcert/releases/download/v1.4.4/mkcert-v1.4.4-linux-amd64
chmod +x mkcert && sudo mv mkcert /usr/local/bin/ # ArgoCD CLI
curl -sSL -o argocd https://github.com/argoproj/argo-cd/releases/latest/download/argocd-linux-amd64
chmod +x argocd && sudo mv argocd /usr/local/bin/ # jq
sudo apt install -y jq # PowerShell 7
sudo apt-get install -y wget && wget -q "https://packages.microsoft.com/config/ubuntu/$(lsb_release -rs)/packages-microsoft-prod.deb"
sudo dpkg -i packages-microsoft-prod.deb && sudo apt-get update && sudo apt-get install -y powershell
```

■ Vérification finale des prérequis

Exécuter ces commandes pour vérifier que tout est bien installé : `docker --version | kubectl version --client | helm version | kind version | argocd version --client | mkcert --version | git --version | jq --version` Si une commande retourne 'not found', revenir à l'étape d'installation correspondante.

Déploiement de l'infrastructure de base : certificats TLS auto-signés avec `mkcert` et `cert-manager`, Ingress NGINX pour le routage HTTP/HTTPS local, et PostgreSQL dans Docker pour la base de données.

En local, on ne peut pas utiliser Let's Encrypt (pas d'IP publique). On utilise mkcert pour créer une CA locale de confiance, puis cert-manager pour gérer les certificats des services.

[illegible][illegible]

Pour simplifier le développement local, PostgreSQL tourne directement dans Docker (pas dans K8s). Il est accessible depuis les pods via un Service ExternalName pointant vers host.docker.internal.

Zéro cloud requis — Tout tourne sur votre machine

■ Sur Linux, `host.docker.internal` n'est pas disponible par défaut. Utiliser : `--add-host=host.docker.internal:host-gateway` dans la commande `docker run` du pod, ou récupérer l'IP gateway avec `'ip route | grep docker0'`.

PARTIE 4 — JFrog Artifactory en Local (Docker)

Déploiement de JFrog Artifactory OSS (version gratuite) en local via Docker. Configuration d'un dépôt Docker local pour stocker les images des microservices ShipClass.

6 Installer JFrog Artifactory OSS (Free) en Local

JFrog Artifactory OSS est la version open source, gratuite et sans limite de fonctionnalités pour les dépôts locaux. Elle tourne parfaitement dans Docker sur votre machine.

Tous OS — Option A : Docker seul (plus simple)

[illegible]

Option B : JFrog dans kind (Helm — recommandé pour cohérence)

```
# █ JFrog via Helm dans le cluster kind ██████████ helm repo add jfrog https://charts.jfrog.io &&
helm repo update helm install artifactory jfrog/artifactory-oss --namespace jfrog --create-namespace --set
artifactory.replicaCount=1 --set artifactory.resources.requests.cpu=200m --set
artifactory.resources.requests.memory=1Gi --set artifactory.persistence.enabled=true --set
artifactory.persistence.size=10Gi --set nginx.enabled=false # On utilisera Ingress NGINX à la place # Exposer via
Ingress avec TLS mkcert kubectl apply -f - <<EOF apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name:
jfrog-ingress namespace: jfrog annotations: cert-manager.io/cluster-issuer: mkcert-issuer
nginx.ingress.kubernetes.io/proxy-body-size: "500m" spec: ingressClassName: nginx tls: - hosts: -
jfrog.shipclass.local secretName: jfrog-tls rules: - host: jfrog.shipclass.local http: paths: - path: / pathType:
Prefix backend: service: name: artifactory port: number: 8082 EOF # Accéder via https://jfrog.shipclass.local
```

7 Configurer JFrog : Dépôt Docker Local + Push d'Images

Tous OS — Configuration via API REST

```
# █ Attendre que JFrog soit prêt ████████████████████████████████████████ $JFROG = "http://localhost:8082" $creds = [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("admin:password")) $headers = @{ Authorization = "Basic $creds"; "Content-Type" = "application/json" } # Vérifier que l'API répond Invoke-RestMethod "$JFROG/artifactory/api/system/ping" -Headers $headers # → "OK" # █ Changer le mot de passe admin ████████████████████████████████████████ $newPw = @{ username="admin"; oldPassword="password"; newPassword="ShipLocal2024!" } | ConvertTo-Json Invoke-RestMethod "$JFROG/artifactory/api/security/users/authorization/changePassword" ` -Method POST -Headers $headers -Body $newPw $headers.Authorization = "Basic " + [Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes("admin:ShipLocal2024!")) # █ Créer un dépôt Docker local ████████████████████████████████████████ $repo = @{ rclass="local"; packageType="docker"; repoLayoutRef="simple-default"; description="Images Docker ShipClass local" } | ConvertTo-Json Invoke-RestMethod "$JFROG/artifactory/api/repositories/docker-shipclass-local" ` -Method PUT -Headers $headers -Body $repo # █ Créer un dépôt Helm local ████████████████████████████████████████ $helm_repo = @{ rclass="local"; packageType="helm"; description="Charts Helm ShipClass" } | ConvertTo-Json Invoke-RestMethod "$JFROG/artifactory/api/repositories/helm-shipclass-local" ` -Method PUT -Headers $headers -Body $helm_repo # █ Configurer Docker pour faire confiance à JFrog local ██████████ # Ajouter dans Docker Desktop → Settings → Docker Engine : # {"insecure-registries": ["localhost:8082"]} # Ou créer /etc/docker/daemon.json : # {"insecure-registries": ["localhost:8082"]} # █ Login Docker → JFrog local ████████████████████████████████████████ docker login localhost:8082 -u admin -p ShipLocal2024! # █ Tester un push d'image ████████████████████████████████████████ docker pull alpine:latest docker tag alpine:latest localhost:8082/docker-shipclass-local/alpine:test docker push localhost:8082/docker-shipclass-local/alpine:test # → Vérifier dans l'UI : http://localhost:8082/ui/repos/browse/local/docker-shipclass-local # █ Secret K8s pour puller depuis JFrog dans kind ████████████████████████████████████████ kubectl create secret docker-registry jfrog-local-secret --docker-server=localhost:8082 --docker-username=admin --docker-password=ShipLocal2024! --namespace=shipclass
```

■ Trivy Scan local (remplace JFrog Xray)

JFrog Xray (scan CVE) est une fonctionnalité Pro payante. En local, utiliser Trivy (gratuit) : `trivy image localhost:8082/docker-shipclass-local/vessel-service:latest --severity HIGH,CRITICAL` Intégrer dans votre pipeline local ou pre-commit hook Git.

Construction des images Docker des microservices, push vers JFrog local ou le registry kind, et déploiement sur le cluster Kubernetes local avec les Secrets, ConfigMaps et Ingress.

Tous OS

[illegible]

Tous OS

```
# Namespace et ConfigMap ██████████ kubectl apply -f - <<EOF apiVersion: v1
kind: Namespace metadata: name: shipclass labels: environment: local --- apiVersion: v1 kind: ConfigMap metadata:
name: shipclass-config namespace: shipclass data: NODE_ENV: development LOG_LEVEL: debug DB_HOST:
postgres-svc.shipclass.svc.cluster.local DB_PORT: "5432" DB_USER: pgadmin JFROG_URL:
http://host.docker.internal:8082/artifactory EOF # █ Secret avec les mots de passe BDD (local = pas de Key Vault) █
kubectl create secret generic shipclass-secrets `--from-literal=DB_PASSWORD=shipclass_local_pw`
--from-literal=JWT_SECRET=local-jwt-secret-dev-only `--from-literal=AIS_API_KEY=mock-ais-key-local`
--namespace=shipclass # █ Deployment vessel-service ██████████ kubectl apply -f -
<<EOF apiVersion: apps/v1 kind: Deployment metadata: name: vessel-service namespace: shipclass spec: replicas: 1
selector: matchLabels: app: vessel-service template: metadata: labels: app: vessel-service spec: containers: - name:
vessel-service image: localhost:5001/vessel-service:latest imagePullPolicy: Always ports: - containerPort: 3001 env: -
name: DATABASE_URL value: "postgresql://\$(DB_USER):\$(DB_PASSWORD)@\$(DB_HOST):\$(DB_PORT)/vessels_db" envFrom: -
configMapRef: name: shipclass-config - secretRef: name: shipclass-secrets resources: requests: cpu: 50m memory: 128Mi
limits: cpu: 300m memory: 256Mi livenessProbe: httpGet: path: /health port: 3001 initialDelaySeconds: 15
periodSeconds: 20 --- apiVersion: v1 kind: Service metadata: name: vessel-svc namespace: shipclass spec: selector:
app: vessel-service ports: - port: 80 targetPort: 3001 EOF # █ Vérifier
██████████ kubectl get pods -n shipclass -w kubectl logs -f
deploy/vessel-service -n shipclass
```

Tous OS

```
# ■■■ Ingress global avec TLS mkcert ██████████ kubectly apply -f <EOF apiVersion:  
networking.k8s.io/v1 kind: Ingress metadata: name: shipclass-ingress namespace: shipclass annotations:  
cert-manager.io/certificate-issuer: mkcert-issuers ingress.kubernetes.io/ssl-redirect: "true"  
nginx.ingress.kubernetes.io/proxy-body-size: "20m" nginx.ingress.kubernetes.io/enabled-cors: "true"  
nginx.ingress.kubernetes.io/cors-allow-origin: "*" spec: ingressClassName: nginx tls: - hosts: - api.shipclass.local -  
shipclass.local secretName: shipclass-tls-secret rules: - host: api.shipclass.local http: paths: - path: /vessels  
pathType: Prefix backend: service: name: vessel-svc port: number: 80 - path: /classifications pathType: Prefix  
backend: service: name: classification-svc port: number: 80 - path: /certificates pathType: Prefix backend: service:  
name: certificate-svc port: number: 80 - path: /inspections pathType: Prefix backend: service: name: inspection-svc  
port: number: 80 - host: shipclass.local http: paths: - path: / pathType: Prefix backend: service: name: frontend-svc  
port: number: 80 EOF # ■■ Test immédiat dans le navigateur ████████████ #  
https://api.shipclass.local/vessels/health → certificat vert █ # https://shipclass.local → frontend React █ # ■■  
Test avec curl ████████████ curl -s  
https://api.shipclass.local/vessels/health # {"status":"ok","database":"connected","version": "1.0.0"}
```

PARTIE 6 — ArgoCD Local : GitOps sur kind

Installation et configuration d'ArgoCD sur le cluster kind local. Utilisation d'un repo Git local (ou GitHub) comme source de vérité. Déploiement automatique des manifests Kubernetes.

11 Installer et Configurer ArgoCD sur kind

Tous OS

```
# ■■■ Installer ArgoCD ■■■ helm repo add argo
https://argoproj.github.io/argo-helm && helm repo update helm install argocd argo/argo-cd `--namespace argocd
--create-namespace `--set server.extraArgs[0]="--insecure" ` # pas de TLS Helm car on utilise Ingress --set
configs.params."server\insecure`=true `--set redis.resources.requests.memory=64Mi `--set
server.resources.requests.memory=128Mi `--set repoServer.resources.requests.memory=128Mi # ■■■ Exposer ArgoCD via
Ingress ■■■ kubectl apply -f - <<EOF apiVersion: networking.k8s.io/v1 kind:
Ingress metadata: name: argocd-ingress namespace: argocd annotations: cert-manager.io/cluster-issuer: mkcert-issuer
nginx.ingress.kubernetes.io/ssl-redirect: "true" nginx.ingress.kubernetes.io/backend-protocol: "HTTP" spec:
ingressClassName: nginx tls: - hosts: - argocd.shipclass.local secretName: argocd-tls rules: - host:
argocd.shipclass.local http: paths: - path: / pathType: Prefix backend: service: name: argocd-server port: number: 80
EOF # ■■■ Récupérer le mot de passe admin ■■■ $pw = kubectl get secret
argocd-initial-admin-secret -n argocd `--o jsonpath='{.data.password}' | ForEach-Object {
[Text.Encoding]::UTF8.GetString([Convert]::FromBase64String($_)) } Write-Host "ArgoCD UI :
https://argocd.shipclass.local" Write-Host "Login : admin" Write-Host "Password : $pw" # ■■■ Login CLI ArgoCD
■■■■ argocd login argocd.shipclass.local `--username admin `
--password $pw `--insecure ` # certificat auto-signé mkcert --grpc-web argocd version # Vérifier la connexion #
argocd: v2.10.x # server: v2.10.x ■■■
```

12 Connecter ArgoCD à un Repo Git Local ou GitHub

Option A — Repo Git Local (gitea dans kind) — 100% offline

```
# ■■ Installer Gitea (serveur Git léger) dans kind ■■■■■■■■■■■■ helm repo add gitea-charts
https://dl.gitea.com/charts && helm repo update helm install gitea gitea-charts/gitea `--namespace gitea
--create-namespace `--set service.http.type=ClusterIP `--set gitea.admin.username=admin `--set
gitea.admin.password=gitea_local_pw `--set gitea.admin.email=admin@shipclass.local `--set persistence.size=2Gi `
--set resources.requests.memory=128Mi ` Ingress pour Gitea kubectl apply -f - <<EOF apiVersion: networking.k8s.io/v1
kind: Ingress metadata: name: gitea-ingress namespace: gitea annotations: cert-manager.io/cluster-issuer:
mkcrt-issuer spec: ingressClassName: nginx tls: - hosts: [gitea.shipclass.local] secretName: gitea-tls rules: - host:
gitea.shipclass.local http: paths: - path: / pathType: Prefix backend: service: name: gitea-http port: number: 3000
EOF # Accéder : https://gitea.shipclass.local (admin/gitea_local_pw) # Créer le repo "k8s-config" dans l'interface #
Connecter ArgoCD à Gitea argocd repo add https://gitea.shipclass.local/admin/k8s-config `--username admin --password
gitea_local_pw `--insecure-skip-server-verification
```

Option B — GitHub (plus simple si connexion internet disponible)

[illegible]

■ Workflow GitOps en local

1. Modifier un fichier YAML dans k8s/ sur votre machine → 2. git commit && git push → 3. ArgoCD détecte le changement (poll toutes les 3 min ou webhook) → 4. ArgoCD applique automatiquement sur le cluster kind → 5. Vérifier dans l'UI <https://argocd.shipclass.local> Pour forcer le sync immédiatement : `argocd app sync shipclass-local --force`

PARTIE 7 — Monitoring Local : Prometheus + Grafana

Déploiement de la stack de monitoring open source Prometheus + Grafana sur le cluster kind. Tableaux de bord préconfigurés pour visualiser les métriques des microservices ShipClass.

13 Installer Prometheus + Grafana sur kind

Tous OS

[illegible]

PARTIE 8 — Aide-Mémoire : Commandes du Quotidien

Toutes les commandes utiles pour gérer l'environnement local au quotidien : démarrer/arrêter le cluster, rebuild les images, accéder aux logs, accéder à la base de données, et dépanner les problèmes courants.

8.1 — Démarrer et Arrêter l'Environnement

Démarrage (à faire chaque matin)

```
# 1. S'assurer que Docker Desktop est lancé docker info # doit répondre sans erreur # 2. Démarrer les conteneurs Docker (JFrog + PostgreSQL) docker start shipclass-postgres docker start shipclass-jfrog # si option Docker seule # 3. Vérifier le cluster kind kubectl get nodes --context kind-shipclass-local # Si les nœuds ne répondent pas, kind redémarre automatiquement avec Docker # 4. Vérifier l'état des pods kubectl get pods -A | Where-Object { $_ -notmatch "Running|Completed" } # Aucune sortie = tout est Running # 5. Ouvrir k9s pour surveiller (optionnel) k9s # Interface TUI - ':pods' pour voir les pods, ':ns' pour les namespaces # Ctrl+C pour quitter
```

Arrêt (fin de journée — libérer la RAM)

```
# Arrêter les conteneurs Docker docker stop shipclass-postgres shipclass-jfrog # kind persiste entre les redémarrages Docker (volumes conservés) # PAS besoin de supprimer le cluster, il redémarre avec Docker Desktop # Si vous voulez libérer TOUTE la RAM (supprime le cluster) # kind delete cluster --name shipclass-local # → Attention : les données Kubernetes sont perdues (pas la BDD PostgreSQL)
```

8.2 — Rebuild et Redéployer un Microservice

Workflow développeur — modifier et tester rapidement

```
# Modifier le code de vessel-service, puis : $SVC = "vessel-service" $REGISTRY = "localhost:5001" # 1. Rebuild de l'image docker build -t "$REGISTRY/$SVC:latest" "services/$SVC/" # 2. Push vers le registry kind docker push "$REGISTRY/$SVC:latest" # 3. Forcer K8s à repuller l'image (rolling restart) kubectl rollout restart deployment/$SVC -n shipclass # 4. Suivre le déploiement kubectl rollout status deployment/$SVC -n shipclass kubectl logs -f deployment/$SVC -n shipclass --tail=50 # Alternative : skaffold (automatise le cycle build/push/deploy) # skaffold dev --port-forward # → Détecte les changements de code et redéploie automatiquement
```

8.3 — Accéder à la Base de Données en Local

Connexion directe à PostgreSQL

```
# Option A : psql direct via Docker docker exec -it shipclass-postgres psql -U pgadmin -d vessels_db # Requêtes utiles : # \l → lister les bases # \dt → lister les tables # \d vessel → décrire la table vessel # SELECT * FROM vessels LIMIT 10; # \q → quitter # Option B : DBeaver / DataGrip / pgAdmin # Host : localhost # Port : 5432 # Database : vessels_db (ou autre) # User : pgadmin # Password : shipclass_local_pw # Option C : Depuis un pod dans le cluster K8s kubectl run pg-client --rm -it --restart=Never --image=postgres:15-alpine -n shipclass -- psql -h postgres-svc -U pgadmin -d vessels_db -W # Password: shipclass_local_pw
```

8.4 — Tableau de Bord Récapitulatif des URLs Locales

Service	URL Locale	Login	Remarque
API ShipClass	https://api.shipclass.local/vessels	—	TLS cert mkcert (vert dans le navigateur)
Frontend	https://shipclass.local	—	Interface React
ArgoCD UI	https://argocd.shipclass.local	admin / voir secret	argocd app list en CLI
JFrog Artifactory	http://localhost:8082	admin / ShipLocal2024!	ou https://jfrog.shipclass.local si Helm
Grafana	https://grafana.shipclass.local	admin / grafana_local_pw	Dashboards K8s préconfigurés
Prometheus	http://localhost:9090	—	port-forward requis

Service	URL Locale	Login	Remarque
PostgreSQL	localhost:5432	pgadmin / shipclass_local_pw	Via DBeaver ou psql
Registry Docker	http://localhost:5001	—	registry kind (pas d'UI)
Gitea (si option A)	https://gitea.shipclass.local	admin / gitea_local_pw	Serveur Git local
k9s (TUI)	Terminal	—	k9s dans le terminal — :pods, :ns

8.5 — Dépannage des Problèmes Courants

Problème	Diagnostic	Solution
Pod en CrashLoopBackOff	<code>kubectl describe pod <pod> -n shipclass</code> <code>kubectl logs <pod> -n shipclass --previous</code>	Vérifier les logs d'erreur. Problèmes fréquents : variable d'env manquante, BDD inaccessible, image introuvable dans le registry.
Certificat TLS non émis (READY=False)	<code>kubectl get certificates -n shipclass</code> <code>kubectl describe challenge -n shipclass</code>	En local avec mkcert, le cert devrait s'émettre en quelques secondes. Si bloqué : <code>kubectl rollout restart deploy/cert-manager -n cert-manager</code>
Ingress retourne 404	<code>kubectl get ingress -n shipclass</code> <code>kubectl describe ingress shipclass-ingress -n shipclass</code>	Vérifier que le service backend existe (<code>kubectl get svc -n shipclass</code>) et que le port correspond.
host.docker.internal non résolu (Linux)	<code>docker inspect shipclass-postgres grep Gateway</code>	Sur Linux, remplacer host.docker.internal par l'IP gateway Docker (ex: 172.17.0.1) dans le Service ExternalName K8s.
ArgoCD ne sync pas	<code>argocd app get shipclass-local</code> <code>argocd app sync shipclass-local --debug</code>	Vérifier que le repo Git est accessible depuis le cluster : <code>argocd repo list</code> . Si erreur TLS : ajouter <code>--insecure-skip-server-verification</code> .
kind sans ressources (OOMKilled)	<code>kubectl top nodes</code> <code>kubectl top pods -n shipclass --sort-by=memory</code>	Réduire les limits mémoire des pods ou augmenter la RAM dans Docker Desktop (Settings → Resources → Memory).
Images introuvables dans le registry kind	<code>curl http://localhost:5001/v2/_catalog</code> <code>docker push localhost:5001/mon-image:latest</code>	Vérifier que kind-registry est connecté au réseau kind : <code>docker network inspect kind grep kind-registry</code> .

■ Environnement local complet opérationnel

Vous avez maintenant un environnement de développement local complet avec : Kubernetes (kind) + Ingress NGINX + TLS auto-signé (mkcert + cert-manager) + PostgreSQL (Docker) + JFrog Artifactory (Docker) + ArgoCD + Prometheus + Grafana. Coût total : 0 € (hors électricité). Temps de démarrage depuis zéro : ~30 minutes. Temps de redémarrage quotidien : ~2 minutes.